Lecture::Serverless and Testing FaaS

Luca Morandini
Head of AI Engineering – Melbourne eResearch Group
University of Melbourne
luca.morandini@unimelb.edu.au

Outline of the Lecture

- **Part 1:** What is FaaS
  - Serverless / FaaS defined
  - Why functions?
  - FaaS services and frameworks

- **Part 2:** More About Functions
  - Functions with side-effects
  - Stateful/stateless functions
  - Synchronous/Asynchronous functions

- **Part 3:** Monolithic vs FaaS Applications

- **Part 4:** Fission FaaS
  - Architecture
  - Fission CLI
  - Fission Specifications

- **Part 5:** Testing FaaS

Part 1: What is Function as a Service (FaaS) ?

Disambiguation

- FaaS is also know as *Serverless computing* (more catchy, but less precise)
- The idea behind Serverless/FaaS is to develop software applications without bothering with the infrastructure (especially scaling-up and down as load increases or decreases): the cloud provider manages servers and capacity
- Therefore, it is more *Server-unseen than Server-less*
- A FaaS environment allows functions to be added, removed, updated, executed, and auto-scaled
- FaaS is, in a way, an extreme form of *microservice architecture*
- In a FaaS environment, functions are executed independently from one another and are loaded into main memory only when required

Why Functions?

- A *function* in computer science is a piece of code that takes in parameters and returns a value

- Functions are the founding concept of *functional programming* - one of the oldest programming paradigms. LISP is one of the oldest (1960) high-level programming languages, and the first to implement the functional paradigm

- Functions are, or should be, *free of side-effects*, *ephemeral*, and *stateless*, which make them ideal for parallel execution and rapid scaling-up and -down,

Functions?

- Function, in the context of FaaS is different from a function in the context of a programming language
- A Python function is a set of statements that may return an object within the same process
- A FaaS function is an independent process that returns a value (often expressed in JSON) consumed by other processes
- A FaaS function, in many FaaS frameworks, is a Docker container

Why FaaS?

- Simpler deployment (the cloud provider takes care of the infrastructure)
- Reduced computing costs (only the time during which functions are executed is billed) and more efficient use of computing resources
- Reduced application complexity and more flexibility due to the intrinsic loosely-coupled architecture

FaaS Applications?

- Functions are triggered by events
- Functions can call each other (function combination) and reused (provided they are generic enough)
- Functions and events can be combined to build software applications
- For instance: a function can be triggered:
  - every hour (say, to compress log files)
  - every time a node is added to a cluster
  - when a pull-request is merged in GitHub
  - when a message is stored in a queue
- Combining event-driven scenarios and functions resembles how User Interface software is built: user actions trigger the execution of pieces of code

FaaS Services and Frameworks

- The first widely available FaaS service was Amazon's *AWS Lambda*. Since then *Google Cloud Functions* (part of Firebase) and *Azure Functions* by Microsoft were introduced

- All of the FaaS above allow functions to use the services of their respective platforms, thus providing a rich development environment

- There are several open-source frameworks (*funtainers* - or *fun*ctions conta*iners*) such as *Apache OpenWhisk*, *OpenFaas*, *Fission*, *Fn*, and *Knative*

- The main difference between proprietary FaaS services and open-source FaaS frameworks is that the latter can be deployed on your cluster, looked into, disassembled, and improved by you.

Part 2: More About Functions

Ephemeral Functions

- An *ephemeral function* is a function whose creation, execution, and removal takes a short period of time

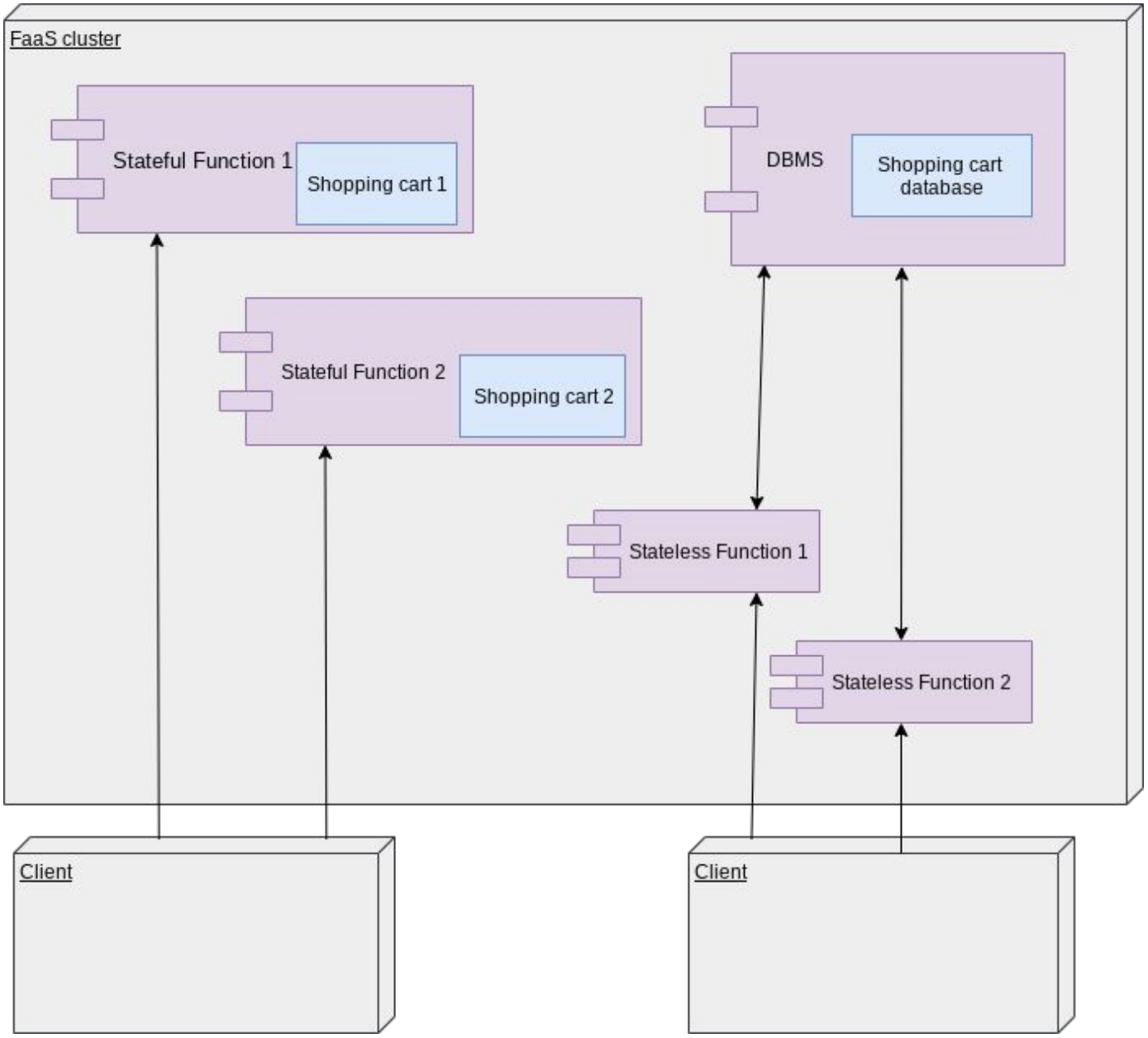
Side-effect-free Functions

- A function that does not modify the state of the software application is said to be *side-effect-free* (for instance, a function that takes an image and returns a thumbnail of that image)
- A function that changes the application is not side-effect-free (for instance, a function that returns a thumbnail of an image that is saved to a database)
- Side-effect-free functions can be run in parallel without worrying much about concurrency issues (such as multiple functions writing the same file at the same time)
- Side effects, however, are inevitable in non-trivial systems. Therefore, consideration must be given on how to make functions with side effects run in parallel, as typically required in FaaS environments, whilst avoiding conflicts on resources
- It is a good practice to limit the number of non-side-effect-free functions to the minimum, rather that scattering the state-altering pieces of code in many functions throughout the software application

Stateful/Stateless Functions #1

- An important subset of functions is the one composed of *stateful functions*
- A stateful function is one whose output changes in relation to internally stored information (hence its output can change from request to request), e.g. a function that adds items to a “shopping cart” and retains that information internally
- Conversely, a *stateless function* is one that does not store information internally, e.g. adding an item to a “shopping cart” stored in a DBMS service and not internally would make the function above stateless, but not side-effect-free
- This is important in FaaS services since there are multiple instances of the same function, and there is no guarantee the same user would call the same function instance twice
- A functions that is stateless and side-effect-free is called a *pure function* (its output depends only on its input)

Stateful/Stateless Functions #2



Synchronous/Asynchronous Functions

- Most FaaS functions are *synchronous*, hence they return their result immediately (or almost so)

- However, there may be functions that take a longer time to return a result, hence they incur timeouts and lock connections with clients in the process, hence it is better to transform them into *asynchronous functions*

- Asynchronous functions return a code that informs the client that the execution has started (usually HTTP status code 202), and then trigger an event when the execution completes

- In more complex cases a *publish/subscribe pattern* involving a message queuing system can be used to deal with asynchronous functions

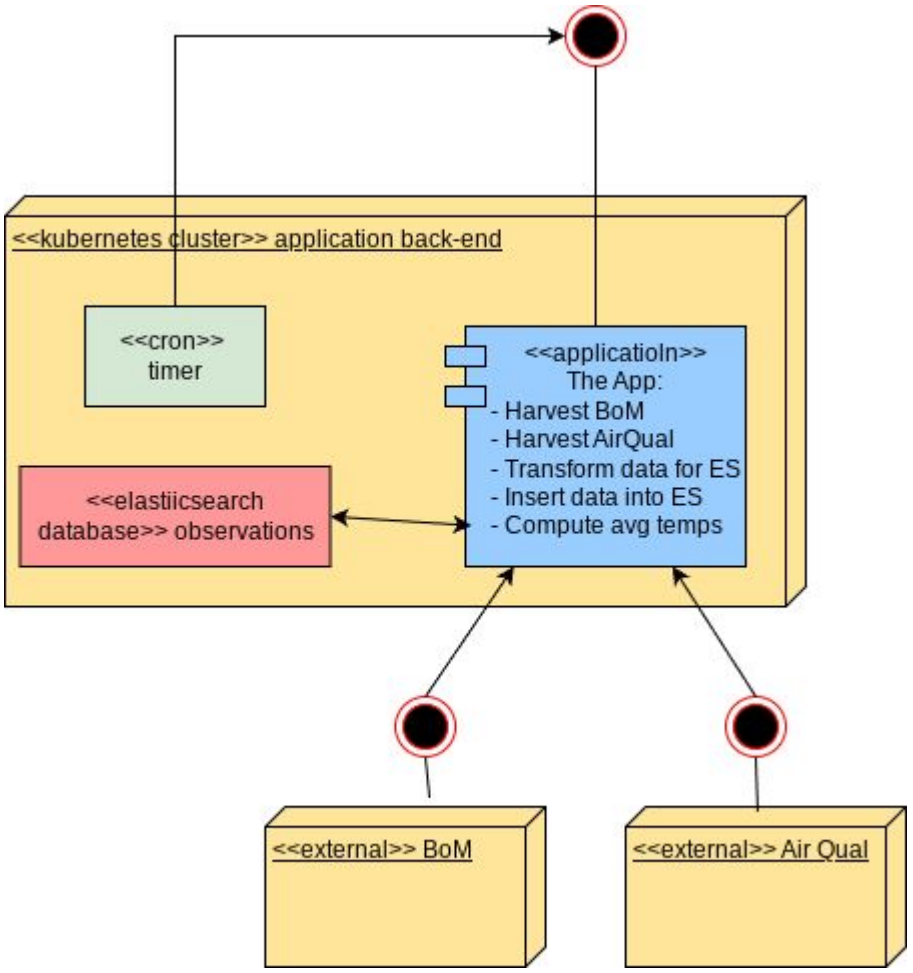
Part 3: Monolithic vs FaaS Applications

The Requirements

- Let’s suppose we have to develop an application that has to:
  - Harvest weather data from a BoM station
  - Harvest pollution data from an Air Quality monitoring service
  - Store data in ElasticSearch
  - Compute the average temperature on a given day for **all** stations
  - Compute the average temperature on a given day for **one** station

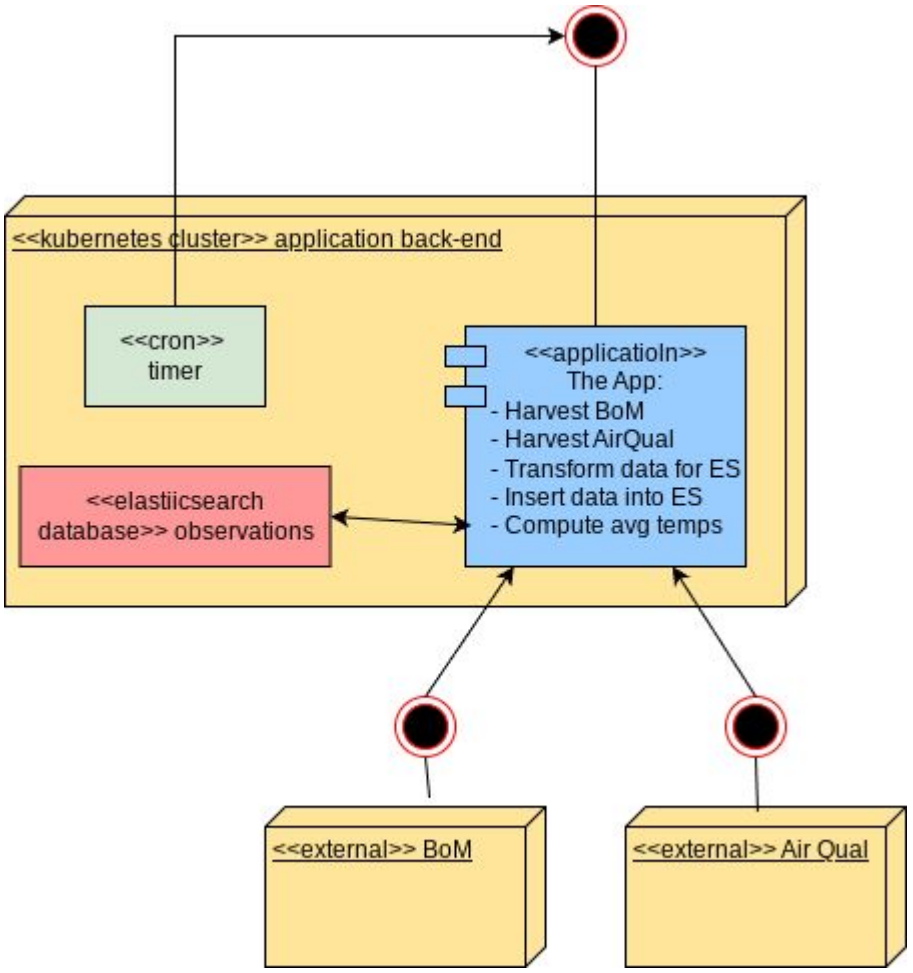
Monolithic Application

- Let's see how a "standard" web-app can satisfy the requirements:
  - A timer calls the harvesting endpoints of the API to trigger data collection from BoM and the Air Quality services
  - Data are converted into a common format
  - The converted data are inserted into ElasticSearch
  - Another endpoint allows a client application to request the average temperatures (computed with an ElasticSearch query)



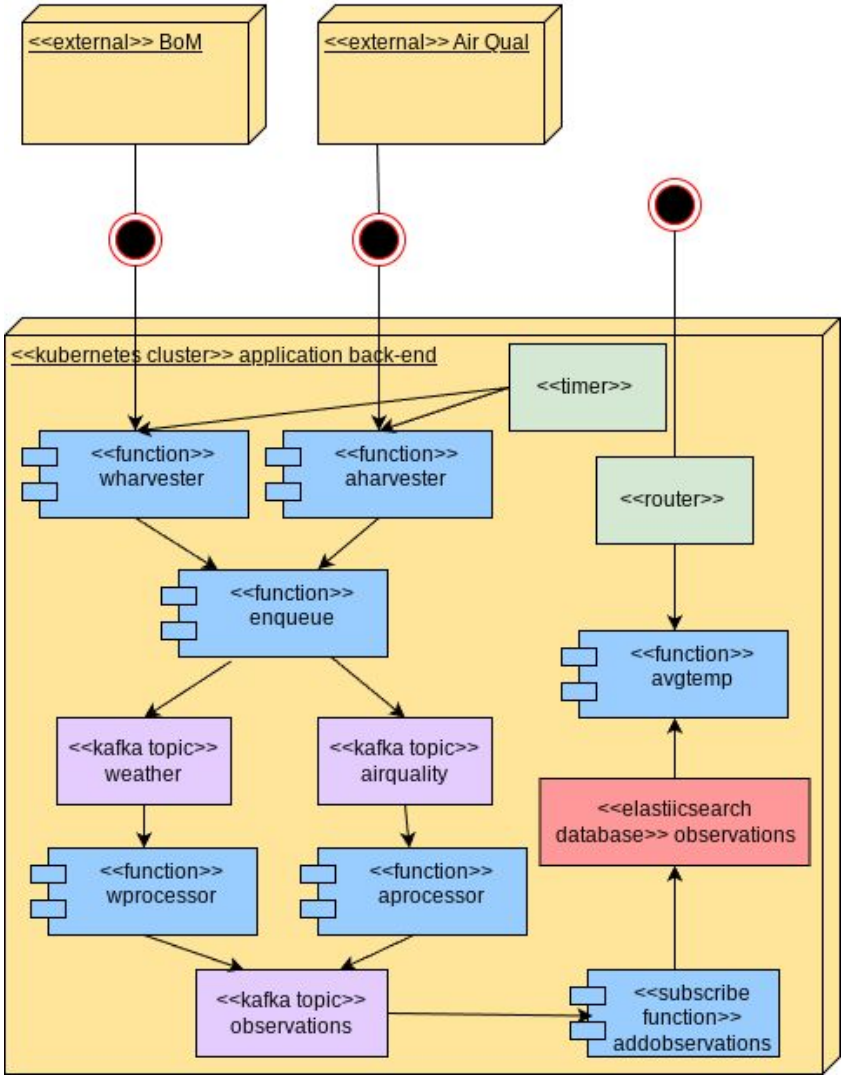
Monolithic Application?

- *The App* can easily scale vertically (with a more powerful VM), but not horizontally (to other nodes in the cluster)
- Most (if not all) modules of *The App* are loaded in memory at the same time leading to more resources used than strictly needed
- If a module of *The App* has a serious malfunction (say an Out of Memory error in Java) it brings down the entire application
- All modules of the application are *tightly-coupled* (most probably written in the same programming language), hence a change to one section of code (say, using a more recent version of a library) may trigger a cascade of other changes



Serverless Version of Application

- Each functionality is delivered by a different (stateless) function
- Most of the functions do not even “talk” to each other directly, but via asynchronous messages in queues
- Functions are loaded only when needed
- A failing function does not bring down the entire application
- Multiple copies of the same function can be distributed on different nodes to scale horizontally
- Functions can be written in different languages and still work together
- Functions can use different environments (e.g. Python 3.9 and Python 3.10) and still work together
- The application is now easier to modify, and more robust



Part 4: Fission FaaS

Fission

- Fission (https://fission.io/) is an open-source framework that runs on Kubernetes and uses Docker containers to deliver FaaS functionality
- Functions can be activated using different triggers (timer, HTTP request, publishing of a message)
- Fission allows to manage deployments either using its CLI or using a declarative application management style (expressed in YAML)
- It can be extended with Argo Workflows (https://argoproj.github.io/workflows/) for function composition

Why Are We Using Fission in This Course?

- It is open-source, hence you can peruse the source code and see how things work, or improve its documentation, raise tickets, solve bugs
- It runs on Kubernetes
- It is relatively simple, unlike Knative, which is a widely used K8s FaaS frameworks
- Deploying functions is quick and simple: by default the developer does not ned to build Docker images and use Docker Registries (which is what happens on Knative, for instance)
- It can use message queues
- It supports many popular programming languages (Python, Node.js, Go, C#, PHP, TensorFlow)

Fission Concepts

- **Function**: a software module that returns a value and can be called independently by a trigger (it runs within a Docker container)

- **Environment**: the Docker image that a *function* runs on. An environment is language-specific, including an HTTP server and some base libraries, but it can be customised with *Packages*

- **Package**: a set of files (usually source code and/or compiled binaries) that is used to customise an *Environment*

- **Trigger**: an event that causes the execution of a *function*, such as:
  - An HTTP request
  - A timer
  - A message published on a queue
  - The completion of a Kubernetes job

- **Router**: a component that directs an HTTP request to a function

- **Specifications (Specs)**: a set of YAML files that defines the components of a Fission application. "*Specs*" make it easier to deploy and maintain applications compared to the use of CLI commands

Environments, Packages, and Functions

- A function runs on a pod, but it is executed in a Docker container within the pod, and that container is based on a Docker image
- The Docker image a function is executed on is called an *environment* in Fission
- The customisation of an environment is done via *package*s attached to a function during deployment
- For instance:
  - Python is an environment (in general you have one environment per programming language in Fission)
  - To add some Python libraries (say, the ElasticSearch client) to the base environment, a *package* has to be created and added to the function in order to enrich the environment during function creation

Function Executors in Fission #1

- Fission executes function in two different ways:
    - **PoolManager**: Fission manages a pool of pods that are always on in each *environment*; when a *function* is invoked the function *packages* are loaded into the pod and the function executed, once the function is completed, the function is removed and the pod is ready to load other functions
    - **NewDeploy**: new pods are created or removed as the load on functions increases or decreases
- Note: the way Fission handles the concurrent execution of functions has changed in the last few versions

Function Executors in Fission #2

- *PoolManager* minimizes the latency in executing functions (*warm start*), but cannot cope with heavy load on a single function

- The function creation parameter `--requestsperpod` (by default 1) allows a function to handle more than one request concurrently, which reduces the number of pods started by the PoolManager

- The function creation parameter `--concurrency` allows to set the maximum number of concurrent pods per function

- *NewDeploy* has a higher latency (*cold start* by default) but allows multiple instances of the same function to always be on, hence it can cope better under heavy load. Additional pods are automatically spun up according to different metrics (such as CPU consumption).

- To reduce latency, a NewDeploy executor can be set to have one function always on (useful when a function is slow when being created because it has to load many libraries and/or large data)

- PoolManager is the default function executor in Fission (for more information about executors in FaaS, see [1])

Autoscaling

- PoolManager scaling can be tailored using Fission own autoscaling options
- The NewDeploy function executor uses a Kubernetes *HPA* (Horizontal Pod Autoscaler), which spawns new pods when a target threshold is passed (say, a pod uses more than a given amount of memory)
- HPA can be used outside Fission to autoscale pods
- *NOTE*: Autoscalers work best with stateless pods
- It is risky to change the number of pods in stateful or non side-effect-free pods, especially in clustered software applications, such as ElasticSearch, that relies on a given number of pods running at all times (Kubernetes StatefulSets are used to meet this specific use case)

The Fission Command Line Interface (CLI)

- Fission has a client *Command Line Interface* that can be used to interact with the cluster
- Differently from Kubectl, but similarly to the OpenStack CLI, Fission CLI uses an *object-action* style (`fission function list`), rather than *action-object* (`kubectl get nodes`)
- For instance, to:
  - list environments: `fission environment list`
  - list packages: `fission packages list`
- Abbreviations for objects can be used:
  - **fn**: Function
  - **env**: Environment
  - **rt**: Route
  - **tr**: Trigger
  - **pkg**: Package
  - **spec/specs**: Specifications
- Some useful commands:
  - To see the log of a function: `fission function log --name <function name> --namespace <namespace>` (yes, k8s namespaces)
  - To invoke a function: `fission function test --name <function name> --namespace <namespace>`
  - To update the cluster based on specs: `fission specs apply`

Writing Functions in Fission

- Crafting a simple function is done easily:
  - Write a Python script in a file named `hello.py`:

```
def main():
    return 'Hello, world!'
```

  - Create the function: `fission function create --name hello --env python --code hello.py`
  - Test the function: `fission function test --name hello`

- The namespace has not been specified in the command, hence the `default` k8s namespace is implied

- When the source code is updated a `fission function update` command must be executed to reflect the changes

- Docker containers can become Fission functions (see `fission function run-container` command, still in Alpha)

- Functions can be listed using `fission function list`, and deleted using `fission function delete`

Invoking Functions in Fission

- While the test command is useful for a quick test, it is not how functions are invoked, for that we need a *trigger*

- The most typical use case is an HTTP request, hence we need to create a *route* (aka *HTTP trigger*):

```
fission route create --name hellort --function hello
--method GET --url /hello
```

- Once a port-forward is established, we can invoke the function from outside the cluster:

```
curl http://localhost:9090/hello
Hello world!
```

- Under the hood, Fission uses *k8s ingresses* to implement routes

- Function can be invoked by other trigger types (publishing of messages on queues, timers, etc)

ReSTful API in Fission

- By adding routers one can create entire ReSTful APIs, including *path parameters* (as `userid` is in "`/users/{:userid}`")
- Let's define a route with a path parameter named "word" defined as a string of alphabetic characters (lowercase or uppercase):

```
fission route update --name hello --function hello --method GET --url
'/hello/{word:[A-Za-z]+}'
```

- Let's change the hello.py function so that it prints the header with the parameter:

```
from flask import request
def main():
    return f'Hello, {request.headers["X-Fission-Params-Word"]}!'
```

- Now the greeting can be changed with any request:

```
curl http://localhost:9090/hello/Luca
Hello, Luca!
```

- The *request parameters* and the *request body* can be accessed using the usual Flask objects, and the path parameters via the headers (see above)
- The same function can be the target of many routes:

```
fission route create --name greetingpost --function hello --method POST
--url '/greeting/{word:[A-Za-z]+}'
fission route create --name hellodelete --function hello --method DELETE
--url '/hello/{word:[A-Za-z]+}'
curl -XPOST http://localhost:9090/greeting/Luca --data '{}'
Hello, Luca!
curl -XDELETE http://localhost:9090/hello/Luca
Hello, Luca!
```

Environments in Fission

- Sometimes additional libraries or source code have to be added to a function
- The way Fission does it by customising environments using packages
- For instance, customising the Python environment usually involves writing (in addition to the function scripts):
  - a `requirements.txt` for dependencies
  - a `build.sh` that executes a pip install command on the container
  - an `__init__.py` (it could be empty)
- The above has to be zipped in an archive and then added to the cluster as a package:

```
fission package create\
  --name mypackage\
  --sourcearchive mypackage.zip\
  --env python\
  --buildcmd './build.sh'
```

- The last step is to create the function with the package (note the *entrypoint*):

```
fission fn create --name myfunction\
  --env python\
  --pkg mypackage\
  --entrypoint "myfunction.main"
```

Fission Archives

- Fission has two types of archives
- Source archives (`--sourcearchive` option) are files that Fission tries to compile and install dependencies on after having unzipped them
- Deploy archives (`--deployarchive` option) are files that Fission merely unzips
- Deploy archives are useful when executable code or data files have to be added to a function

Other Triggers

- Fission functions can be invoked at intervals using timers:

```
fission timer create\
 --name everyminuteMyFunction\
 --function myfunction\
 --cron "@every 1m"
```

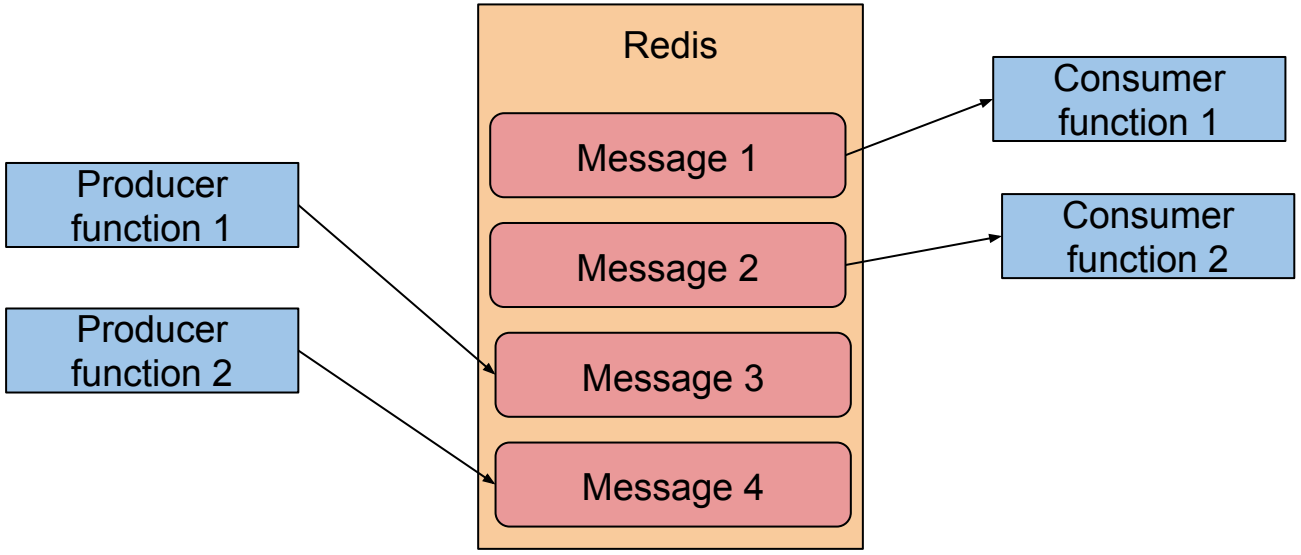
- NOTE: timer intervals must be longer than the function run time, to avoid an ever-increasing number of functions running

- Fission functions can be invoked as well when there are changes in Kubernetes resources (pods, ingresses, etc)

```
fission watch create\
 --name podEventsMyFunction\
 --function myfunction\
 --type pod
```

Fission Message Queues

- Fission can be integrated with messages queues, so that messages can be stored by a function and later consumed by another function
- This way asynchronous functions can be implemented (for instance, a function can store a message in a queue for another function to pick up at a later time)
- **Queues** have to be backed-up by a persistence mechanism, such as Apache Kafka or Redis (which are fast stores that can handle heavy loads and still have good performance)



Fission Websockets

- Long-running tasks are best served by asynchronous functions, but this does not sit well with the synchronous nature of HTTP requests

- Suppose we have a JavaScript client that is used to start a Machine Learning task that may take several minutes; a possible solution may look like this:
  - The client sends an HTTP POST request
  - Fission stores the task request into a queue and returns HTTP status 202 (which means *request accepted*)
  - The client opens a *websocket* (which is a two-way, non-blocking, channel between client and server)
  - A Fission function takes the request from the queue and executes it
  - Once the function has completed its task, the websocket channel is used to send the results back to the client

- This way Fission can accommodate long-running tasks between a back-end application and a web front-end

- Websockets are available in the Node.js environment (see [2])

Fission Specifications

- Using the Fission CLI is not very efficient, as a developer has to issue a command every time a component (a function, a route, etc) is changed
- In general, issuing commands goes against two key principles (which we introduced in the Kubernetes lecture): Infrastructure as Code and Declarative Application Management
- The solution to this is the use *specifications (or specs)*: a bunch of YAML files under a directory that can be applied on the cluster in one go with `fission specs apply` (by default the directory is named *specs*)
- All the options of fission commands have a counterpart in a YAML file, therefore specs and Fission CLI commands are equally powerful
- Instead of executing a command, one may write the equivalent YAML file by using the `--spec` option, which makes it easy to populate the specs directory
- You can sync the resources in the specs directory with the resources on the cluster with `fission specs apply --delete` (it removes from the cluster the resources that are not in the specs)
- The command `fission specs destroy` eliminates from the cluster all the resources that are in the specs directory

Passing Parameters to Fission Functions

- As you may recall from the Kubernetes lecture, a handy way to circulate parameters in a cluster is by using *configmaps*

- Kubernetes uses this feature to set environment variables or files within the pods without explicitly passing their values:

```
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: default
  name: shared-data
data:
  ES_USERNAME: elastic
  ES_PASSWORD: elastic

kubectl apply -f shared-data.yaml
```

(of course, using credentials in the clear is not a good idea -k8s has *secrets* for this use case- but this is just an example):

- This is how to use them in Python (the values of the variables are written as files in a directory with the same namespace and name of the ConfigMap):

```
def config(k):
    with open(f'/configs/default/shared-data/{k}', 'r') as f:
       return f.read()
```

- Fission uses files because ConfigMaps values can hold complex types (say, JSON objects) and they are not just limited to numbers and strings

Fission Functions Timeouts

- Fission has different timeouts that can be set when creating a function
- `--specializationtimeout` function creation timeout (120-second default)
- `--fntimeout` function execution timeout (60-second default)
- `--idletimeout` function idle timeout before recycling (120-second default)
- Other timeouts (ingress controllers, proxies, etc) may apply as well

Part 5: Testing FaaS

How to Test FaaS?

- FaaS is no different from other software and can be tested just as well
- In the following slides and accompanying software I will build step-by-step a toy FaaS application with a ReSTful API and its test software. This will you give some examples on how to develop in Fission, with ElasticSearch and ReSTful APIs
- I will develop the application code and the test code in four different *iterations*, simulating the work of a development team
- The source code is available in the comp90024 GitLab repo we use in other workshops, under the `test` directory
- The test source code is kept intentionally plain (minimal exception handling, verbose coding, and with no use of functional programming constructs)
- But first, a quick recap on the best practices on how to test software

Why We Test?

- Because it saves time and improves team dynamics
- Every software is prone to malfunctions: sometimes for bad coding practices (missing exception handlings), sometimes for lack of imagination (systems can fail in so many unexpected ways), more often than not software fails because it was not tested properly
- The later you catch a bug the more expensive (in time and effort) it is to find the cause and solve it
- Finding a bug late in the development process often causes friction within the team (“it’s the user interface”, “no, it’s the database”, etc), which is time consuming as well

How We Test?

- **Automatically**: tests have to run automatically, so that there is no excuse to avoid running them for lack of time

- **Early**: writing the test concurrently with software design allows to catch design flaws early on, saving time (*Test-Driven Development* [3])

- **Comprehensively**: tests must cover the whole system. Testing single modules but not the interactions between modules delays the discovery of issues and makes their resolution harder (splitting the system into parts to be done separately by different team members and expecting all the parts to fit together ar the end almost never works)

- **Incrementally (code a little, test a little)**: run tests whenever a change is made ensure that no new defects were introduced as a result of the change (a change in a library version may make a module work but another fail)

What We Test?

- **Expected conditions**: we test the software for success under expected conditions

- **Boundary conditions**: we test the software for success under expected but unusual conditions (does the system work if a variable is set to 0, or a ‘division by 0’ error is raised instead?)

- **Error conditions**: we test the software for failure of other components (does the system return a correct error message when the database is not reachable?)

Test Taxonomy

- **Unit tests**: test individual individual modules in isolation

- **Integration tests**: test how the software integrates with other components, for instance a database. To test for error conditions, a *mock* may be used. (A *mock* is a piece of software that is used to simulate another component, such as a database, so that error conditions can be simulated at will)

- **System tests (end-to-end tests)**: test how the software behaves as a whole (as a client would see it)

Toy Application Requirements

- Add a student to database
- Remove a student from a database
- Update student data
- Return data about a student
- Add a course to a database
- Remove a course from a database
- Update course data
- Return data about a course
- Produce a list of students
- Produce a list of students given a course
- Produce a list of courses

ReSTful API Design

- Let's start by designing the API
- The URL space can be designed as:
  - `/students (GET)`
  - `/students/{studentid:} (GET, PUT, DELETE)`
  - `/courses (GET)`
  - `/courses/{courseid:} (GET, PUT, DELETE)`
  - `/courses/{courseid:}/students (GET)`
- The application will be written in Python, using Fission and ElasticSearch
- The application will be deployed in the `default` Kubernetes namespace
- The application relies on a python environment created as:
  `fission env create --spec --name python --image fission/python-env --builder fission/python-builder`
  (Remember to initialize the `specs` directory with `fission specs init`)

First Iteration - Functions

- Let's write down *dummy functions*, such as `course`:

```
from flask import request, current_app
import requests, logging, json
def main():
    current_app.logger.debug(f'Received request: {request.method}')
    return 'course', 200

fission function create --name course --spec --env python --code
./functions/course/course.py
```

- …and so on for `student`, `courses`, `students`, and `coursesstudents`

- The specs can then be applied with `fission spec apply`

First Iteration - Routes

- Let's write the routes we need

```
fission route create --spec --name studentget --function student --url
'/students/{studentid:[0-9]+}' --method GET
fission route create --spec --name studentput --function student --url
'/students/{studentid:[0-9]+}' --method PUT
fission route create --spec --name studentdelete --function student --url
'/students/{studentid:[0-9]+}' --method DELETE

fission route create --spec --name courseget --function course --url
'/courses/{courseid:[0-9]+}' --method GET
fission route create --spec --name courseput --function course --url
'/courses/{courseid:[0-9]+}' --method PUT
fission route create --spec --name coursedelete --function course --url
'/courses/{courseid:[0-9]+}' --method DELETE

fission route create --spec --name students --function students --url
'/students' --method GET
fission route create --spec --name courses --function courses --url
'/courses' --method GET

fission route create --spec --name coursesstudents --function
coursesstudents --url '/courses/{courseid:[0-9]+}/students' --method GET
```

…and then apply the specs:

```
fission specs apply
```

First Iteration - End-to-End Tests

- The simplest e2e test possible (using the python unittets package):

```
class TestEnd2End(unittest.TestCase):
   def test_student(self):
       self.assertEqual(test_request.get('/students/111').status_code, 200)
       self.assertEqual(test_request.get('/students/111').text, 'student')

       self.assertEqual(test_request.put('/students/111', {}).status_code, 200)
       self.assertEqual(test_request.put('/students/111', {}).text, 'student')

       self.assertEqual(test_request.delete('/students/111').status_code, 200)
       self.assertEqual(test_request.delete('/students/111').text, 'student')

   def test_students(self):
       self.assertEqual(test_request.get('/students').status_code, 200)
       self.assertEqual(test_request.get('/students').text, 'students')

..
```

- After having set the port forward of the application to `9090` on localhost, we can run the tests (`kubectl port-forward service/router -n fission 9090:80`)

- Executing `python tests/end2end.py` should return success

- This is just a test to check whether the routes are there, the functions have been deployed, the right functions are called by the routes, and the functions return 200

Second Iteration: Parameters

- During the first iteration we added dummies functions and routes, but, more importantly, we tested them and know that they work… time to be more ambitious

- Let's set global parameters using a k8s ConfigMap:

```
apiVersion: v1
kind: ConfigMap
metadata:
  namespace: default
  name: parameters
data:
  ES_URL:  'https://elasticsearch-master.elastic.svc.cluster.local:9200'
  ES_USERNAME: 'elastic'
  ES_PASSWORD: 'elastic'
  ES_DATABASE: 'students'
```

```
kubectl apply -f specs/configmap-parameters.yaml
```

(Credentials must not be shared unencrypted, obviously. Use Kubernetes secrets instead)

- The configmap has to be added to the functions YAML files in the specs directory

```
configmaps:
  - name: parameters
    namespace: default
```

Second Iteration: Database

- Let’s create a database with students and courses in ElasticSearch after having forwarded the ElasticSearch port:

```
kubectl port-forward service/elasticsearch-master -n elastic 9200:9200
curl -XPUT -k 'https://127.0.0.1:9200/students' \
   --user 'elastic:elastic' \
   --header 'Content-Type: application/json' \
   --data '{
      "settings": {
          "index": {
              "number_of_shards": 3,
              "number_of_replicas": 1
          }
      },
      "mappings": {
          "properties": {
              "id": {
                  "type": "keyword"
              },
              "type": {
                  "type": "keyword"
              },
              "timestamp": {
                  "type": "date"
              },
              "name": {
                  "type": "keyword"
              },
              "courses": {
                  "type": "keyword"
              }
          }
      }
    }'
```

Second Iteration: Functions

- Let's check whether a Fission function can read from the database using the parameters shared using the ConfigMap:

```
from flask import request, current_app
import requests, logging, json

def config(k):
    with open(f'/configs/default/parameters/{k}', 'r') as f:
       return f.read()

def main():
    r = requests.get(f'{config("ES_URL")}/{config("ES_DATABASE")}',
            verify=False,
            auth=(config("ES_USERNAME"), config("ES_PASSWORD")))
    return r.json(), r.status_code
```

- The tests now check whether a JSON is returned:

```
def test_students(self):
     self.assertEqual(test_request.get('/students').status_code, 200)
     self.assertIsNotNone(test_request.get('/students').json())
```

- It is now starting to look like a real application: it queries the database and returns something from it, and we can still prove (with our end-to-end tests) that it works… time to do something more with database data

Third Iteration - Side Effects

- So far the functions under test have been *side-effect-free*, which is not what we want to test, since the requirements call for transactions (adding students to the database, deleting courses from the database, etc)
- To test transactions we need a database that we can wipe and fill with data at will: a t*est database*
- To keep things simple, the same “students” database will be used (<u>not</u> good practice, but ok for a demonstration)
- Let’s write a function that re-creates the database for testing purpose (also, this comes in handy to document the database structure in code)
- Before that, let’s add the database schema as a parameter to our ConfigMap

```
…
data:
  ES_URL: 'https://elasticsearch-master.elastic.svc.cluster.local:9200'
  ES_USERNAME: 'elastic'
  ES_PASSWORD: 'elastic'
  ES_DATABASE: 'students'
  ES_SCHEMA: |
    {
      "settings": {
          "index": {
              "number_of_shards": 3,
              "number_of_replicas": 1
    …
```

We will go through shards/replicas next week in ElasticSearch

- The ConfigMap has to be updated on the cluster

```
kubectl apply -f specs/configmap-parameters.yaml
```

Third Iteration - Wipe Test Database

- The function uses the `ES_SCHEMA` parameter we just created to delete and recreate an ElasticSearch database

```
def main():
    r = requests.delete(f'{config("ES_URL")}/{config("ES_DATABASE")}',
            verify=False,
            auth=(config("ES_USERNAME"), config("ES_PASSWORD")))
    r = requests.put(f'{config("ES_URL")}/{config("ES_DATABASE")}',
            verify=False,
            auth=(config("ES_USERNAME"), config("ES_PASSWORD")),
            data=json.loads(config("ES_SCHEMA")))
    return r.json(), r.status_code
```

- We then have to create function and route, manually add the configmap to the function spec just created, and then apply the specs

```
fission function create --name wipedatabase --spec --env python --code
./functions/wipedatabase/wipedatabase.py
```

(Manual edit of the `function-wipedatabase.yaml` file)

```
fission route create --spec --name wipedatabase --function wipedatabase --url
'/wipedatabase' --method DELETE
fission specs apply
```

- Time to test the wipedatabase function

```
curl -XDELETE 'http://127.0.0.1:9090/wipedatabase'
```

Getting:

```
{"acknowledged":true,"index":"students","shards_acknowledged":true}
```

Third Iteration - Tests #1

- Test cases for transactions with database wipe before each test

```
…
def setUp(self):
    self.assertEqual(test_request.delete('/wipedatabase').status_code, 200)
    time.sleep(1)

def test_student(self):
    self.assertEqual(test_request.put('/students/1', {'name': 'John Doe',
'courses': '90024'}).status_code, 201)
    self.assertEqual(test_request.put('/students/2', {'name': 'Jane Doe',
'courses': ['90024', '90059']}).status_code, 201)
    time.sleep(1)

    r= test_request.get('/students/1')
    self.assertEqual(r.status_code, 200)
    o= r.json()['_source']
    self.assertEqual(o['name'], 'John Doe')
    self.assertEqual(o['courses'], '90024')

    r= test_request.get('/students/2')
    o= r.json()['_source']
    self.assertEqual(r.status_code, 200)
    self.assertEqual(o['name'], 'Jane Doe')
    self.assertEqual(o['courses'], ['90024', '90059'])
…
```

Third Iteration - Tests #2

```
    …
    r= test_request.put('/students/1', {'name': 'Bob Carr', 'courses':
'90024'})
    self.assertEqual(r.status_code, 201)
    r= test_request.get('/students/1')
    o= r.json()['_source']
    self.assertEqual(r.status_code, 200)
    self.assertEqual(o['name'], 'Bob Carr')
    self.assertEqual(o['courses'], '90024')

    self.assertEqual(test_request.get('/students/999').status_code, 404)

    self.assertEqual(test_request.delete('/students/1').status_code, 200)
    self.assertEqual(test_request.delete('/students/2').status_code, 200)
```

- Obviously the tests fail, as they should because there is no code in the functions to change the database
- It's good practice to have a test case fail before making it work by changing the application (in this way you are sure the test works in either case: success or fail)
- You may notice there is one test case testing error conditions (404)

Third Iteration - Implement Transactions

- `student` function that changes the database:

```
…
def docurl():
      return
f'{config("ES_URL")}/{config("ES_DATABASE")}/_doc/student_{request.headers["X-Fission-Params-Studentid"]}'

def addfields(data):
    data['timestamp'] = datetime.now().isoformat()
    data['id'] = request.headers['X-Fission-Params-Studentid']
    data['type'] = 'student'
    return data

def main():
    try:
        if request.method == 'GET':
            r = requests.get(docurl(), verify=False, auth=(config("ES_USERNAME"), config("ES_PASSWORD")))
            return r.json(), r.status_code
        elif request.method == 'PUT':
            r = requests.get(docurl(), verify=False, auth=(config("ES_USERNAME"), config("ES_PASSWORD")))
            if r.status_code == 200:
                params={'if_seq_no': r.json()['_seq_no'], 'if_primary_term': r.json()['_primary_term']}
            else:
                params= {}
            r = requests.put(docurl(), verify=False, auth=(config("ES_USERNAME"), config("ES_PASSWORD")),
                    headers={'Content-type': 'application/json'},
                    data=json.dumps(addfields(request.json)),
                    params=params)
            return r.json(), r.status_code
        elif request.method == 'DELETE':
            r = requests.get(docurl(), verify=False, auth=(config("ES_USERNAME"), config("ES_PASSWORD")))
            if r.status_code != 200:
                return r.json(), r.status_code
            r = requests.delete(docurl(), verify=False, auth=(config("ES_USERNAME"), config("ES_PASSWORD")),
                    params={'if_seq_no': r.json()['_seq_no'], 'if_primary_term': r.json()['_primary_term']})
            return r.json(), r.status_code
    except Exception as e:
        current_app.logger.error(e)
        return {'message': f'Error {e}'}, 500
    return {'message':'Method not allowed'}, 405
```

Third Iteration - Queries

```
def main():
    try:
        r = requests.post(f'{config("ES_URL")}/{config("ES_DATABASE")}/_search',
                verify=False,
                auth=(config('ES_USERNAME'), config('ES_PASSWORD')),
                headers={'Content-type': 'application/json'},
                data=json.dumps({'_source': False,
                    'query': {
                        'term': {
                            'type': {
                                'value': 'student'
                            }
                        }
                    },
                    'fields':[
                        {'field':'id'},
                        {'field':'timestamp'},
                        {'field':'name'},
                        {'field':'courses'}
                    ],
                    'sort': [
                        {
                          'name': {
                            'order': 'asc',
                            'missing': '_last',
                            'unmapped_type': 'keyword'
                          }
                       }
                    ]
                    })
                )
        return r.json(), r.status_code

    except Exception as e:
        current_app.logger.error(e)
        return {'message': f'Error {e}'}, 500

    return {'message':'Method not allowed'}, 405
```

Third Iteration - Test Queries

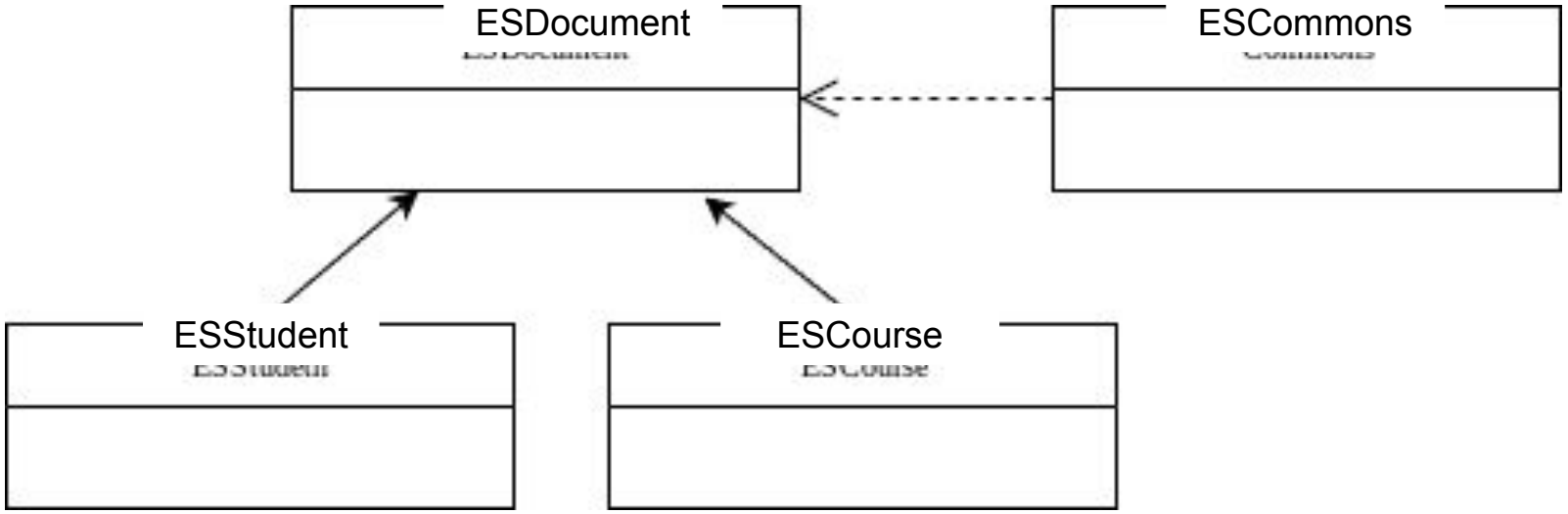
```
def test_students(self):
    self.assertEqual(test_request.put('/courses/90024', {'name': 'Cloud Computing'}).status_code, 201)
   self.assertEqual(test_request.put('/courses/90059', {'name': 'Introduction to Programming'}).status_code,
201)
   self.assertEqual(test_request.put('/students/1', {'name': 'John Doe', 'courses': '90024'}).status_code,
201)
 self.assertEqual(test_request.put('/students/2', {'name': 'Jane Doe', 'courses': ['90024',
'90059']}).status_code, 201)
    time.sleep(1)

    r= test_request.get('/students')
    o= (r.json()['hits'])['hits']
    self.assertEqual(r.status_code, 200)
    self.assertEqual(o[0]['fields']['name'][0], 'Jane Doe')
    self.assertEqual(o[1]['fields']['name'][0], 'John Doe')
```

Fourth Iteration - Refactoring #1

- The application is working since the end-to-end tests work…
  …but the codebase is bloated, with duplications of code and a general lack of abstracting common behaviour

- The answer is refactoring (restructuring the code while keeping its current behaviour)

- For instance, we could restructure the software using 4 classes (see this UML Class Diagram):



Fourth Iteration - Refactoring #2

- Some common behaviour is grouped in the Commons class

```
class Commons:

    @staticmethod
    def config(k):
        with open(f'/configs/default/parameters/{k}', 'r') as f:
            return f.read()

    @staticmethod
    def auth():
        return (Commons.config("ES_USERNAME"), Commons.config("ES_PASSWORD"))

    @staticmethod
    def search_url():
        return f'{Commons.config("ES_URL")}/{Commons.config("ES_DATABASE")}/_search'
```

Fourth Iteration - Refactoring #3

- Another class could hold the behaviour common to students and courses

```
…
class ESDocument:
    def __init__(self, commons, req, type):
        self.commons = commons
        self.req = req
        self.type = type

    def url(self):
      return
f'{self.commons.config("ES_URL")}/{self.commons.config("ES_DATABASE")}/_doc/{self.type}_{self.req.headers[f"X
-Fission-Params-{self.type.capitalize()}id"]}'

    def add_fields(self, data):
        data['timestamp'] = datetime.now().isoformat()
        data['id'] = self.req.headers[f'X-Fission-Params-{self.type.capitalize()}id']
        data['type'] = self.type
        return data

    def get(self):
        r = requests.get(self.doc_url(), verify=False, auth=self.commons.auth())
        return r.json(), r.status_code

    def put(self):
        r = requests.get(self.doc_url(), verify=False, auth=self.commons.auth())

        if r.status_code == 200:
            params={'if_seq_no': r.json()['_seq_no'], 'if_primary_term': r.json()['_primary_term']}
        else:
            params= {}

        r = requests.put(self.doc_url(), verify=False, auth=self.commons.auth(),
                headers={'Content-type': 'application/json'},
                data=json.dumps(self.add_fields(request.json)),
                params=params)
        return r.json(), r.status_code

    def delete(self):
        r = requests.get(self.doc_url(), verify=False, auth=self.commons.auth())
        …
```

Fourth Iteration - Refactoring #4

- The classes for course and document are just a specialization of the base class

```
from ESDocument import ESDocument

class ESCourse(ESDocument):

    def __init__(self, commons, req):
        super(ESCourse, self).__init__(commons, req, 'course')
```

- The function `wipedatabase` now could then be slimmed down to something like:

```
from Commons import Commons

def main():
    r = requests.delete(f'{Commons.config("ES_URL")}/{Commons.config("ES_DATABASE")}',
            verify=False,
            auth=Commons.auth())
    r = requests.put(f'{Commons.config("ES_URL")}/{Commons.config("ES_DATABASE")}',
            verify=False,
            auth=Commons.auth(),
            headers={'Content-type': 'application/json'},
            data=Commons.config("ES_SCHEMA"))

    return r.json(), r.status_code
```

- Such a radical refactoring would be risky… but the end-to-end tests that we wrote assure us that the application is still working as expected

Fourth Iteration - Refactoring #5

- Adding source code dependencies to a Fission function requires changes to the function definition in the specs directory

```
include:
  - ./functions/wipedatabase/wipedatabase.py
  - ./functions/library/Commons.py
kind: ArchiveUploadSpec
name: functions-wipedatabase-wipedatabase-py-1WkK

---
apiVersion: fission.io/v1
kind: Package
metadata:
  creationTimestamp: null
  name: wipedatabase-1a9262b4-002e-48ce-bdb0-cb5400ea777d
…
---
apiVersion: fission.io/v1
kind: Function
metadata:
  creationTimestamp: null
  name: wipedatabase
spec:
…
  package:
    functionName: wipedatabase.main
    packageref:
      name: wipedatabase-1a9262b4-002e-48ce-bdb0-cb5400ea777d
      namespace: default
  requestsPerPod: 1
  resources: {}
```

Fourth Iteration - Unit Tests

- It is never too late to add some unit tests

```
class TestUnit(unittest.TestCase):

    def setUp(self):
        self.mock_commons = MagicMock(autospec=Commons)
        self.mock_commons.config = MagicMock(return_value='x')
        self.mock_request = MagicMock(autospec=request)
        self.mock_request.headers ={'X-Fission-Params-Courseid': 'comp90024',
'X-Fission-Params-Studentid': '123'}

    def test_escourse(self):
        test_escourse= ESCourse(self.mock_commons, self.mock_request)
        self.assertEqual(test_escourse.url(), 'x/x/_doc/course_comp90024')

    def test_esstudent(self):
        test_esstudent= ESStudent(self.mock_commons, self.mock_request)
        self.assertEqual(test_esstudent.url(), 'x/x/_doc/student_123')
```

In Closing

- Unit tests test individual functions and classes
- NOT COVERED HERE: Integration tests test error conditions by simulating components (such as the DBMSs).
- End-to-end tests test the entire system
- Test before code (Test-Driven Development), because:
  - Writing test cases in advance allow to better understand the requirements
  - A robust set of end-to-end test cases allow to refactor the code with confidence
  - Putting together all the modules, even at a prototypical stage, allow to spot flaws in the software design early on
- More in general: you want to know the bad news early, so that you have more time to fix issues
- It may be a good idea to have a team member in charge of writing the end-to-end test code to nudge the other members into Test-Driven Development

What Must Not Be Done (But I Did Anyway)

- A test database must be run separately from other databases
- Credentials must be stored in secrets, not ConfigMaps
- Integration tests must have been developed to test for conditions that cannot be replicated easily (DBMS becoming inaccessible, malformed JSON returned by the DBMS, etc) - this would have meant implementing a mock DBMS endpoint, outside of the scope of this lecture
- The database schema (mappings) is not ideal (same schema used for courses and students, for instance)... but then ElasticSearch is not ideal for transactions
- The `/wipedatabase` route used for testing must not be publicly available
- The application should not return the ElasticSearch response as it is
- The application should check the schema of incoming JSON objects
- The application does not paginate the results of searches
- The application is only a teaching tool, do not deploy it

Bibliography

[1] *Open Source FaaS Performance Aspects, D. Balla et al., 2020*

[2] https://fission.io/blog/fission-websocket-sample/

[3] *Test Driven Development: By Example*, Kent Beck, 2002